

Comparisons – Simple Hinglish Summary

Source: <https://javascript.info/comparison>

Main baat

Comparison operators maths jaise hi hain, lekin JavaScript me kuch **quirks** (ajeeb baatein) hain. Chapter ke end me unse bachne ka tarika bhi hai.

Comparison operators

Operator	Matlab
<code>a > b</code> , <code>a < b</code>	Bada / chhota
<code>a >= b</code> , <code>a <= b</code>	Bada ya barabar / chhota ya barabar
<code>a == b</code>	Barabar hai? (double <code>=</code>)
<code>a != b</code>	Barabar nahi hai

Note: `==` compare karta hai, aur single `=` assignment hai.

1. Result hamesha Boolean

Har comparison `true` ya `false` deta hai:

```
javascript

alert( 2 > 1 ); // true
alert( 2 == 1 ); // false
alert( 2 != 1 ); // true

let result = 5 > 4; // variable me store bhi kar sakte hain
alert( result ); // true
```

2. String comparison

Strings ko **dictionary (lexicographical) order** me letter-by-letter compare kiya jata hai.

```
javascript

alert( 'Z' > 'A' ); // true
alert( 'Glow' > 'Glee' ); // true
alert( 'Bee' > 'Be' ); // true
```

Algorithm:

1. Dono strings ke pehle character compare karo.
2. Jiska character bada, wo string badi. Kaam khatam.
3. Barabar ho to agla character compare karo.
4. Ye tab tak karo jab tak kisi ek string ka end na aa jaye.
5. Dono ek saath khatam hon to barabar, warna **lambi string badi**.

Dhyan: Ye asli dictionary nahi, **Unicode order** hai. **Case matter karta hai:** `"a"` (lowercase) **bada** hota hai `"A"` (uppercase) se, kyunki Unicode me lowercase ka index bada hota hai.

3. Alag types ka comparison

Alag types compare karne par JavaScript unhe **numbers me convert** kar deta hai:

javascript

```
alert( '2' > 1 );    // true  ('2' → 2)
alert( '01' == 1 );  // true  ('01' → 1)
```

Boolean me `true` → `1` aur `false` → `0`:

javascript

```
alert( true == 1 );  // true
alert( false == 0 ); // true
```

Ajeeb natija

Ek hi time par do values **barabar** bhi ho sakti hain, aur ek `true` aur dusri `false` boolean bhi:

javascript

```
let a = 0;
alert( Boolean(a) ); // false

let b = "0";
alert( Boolean(b) ); // true

alert( a == b );    // true!
```

Kyunki `==` **numeric conversion** karta hai (`"0"` → `0`), jabki `Boolean()` ke rules alag hain.

4. Strict equality (`===`)

Normal `==` ki problem: wo `0` aur `false` me **fark nahi kar sakta**:

javascript

```
alert( 0 == false ); // true
alert( '' == false ); // true
```

Kyunki alag types ko `==` number me convert kar deta hai (empty string aur `false` dono `0` ban jate hain).

`===` (strict equality) bina type conversion ke check karta hai. Types alag hon to seedha `false`:

javascript

```
alert( 0 === false ); // false
```

Iska ulta hai `!==` (strict not-equal).

Tip: `===` likhna thoda lamba hai, lekin code saaf rehta hai aur galti kam hoti hai.

5. `null` aur `undefined` ke saath comparison

Yahan behavior thoda ulta-seedha hai.

Check	Result
<code>null === undefined</code>	<code>false</code> (alag types)
<code>null == undefined</code>	<code>true</code> (special rule: ye dono sirf ek dusre ke barabar hain)
<code>></code> , <code><</code> , <code>>=</code> , <code><=</code> ke saath	<code>null</code> → <code>0</code> , <code>undefined</code> → <code>NaN</code>

Ajeeb result: `null` vs `0`

javascript

```
alert( null > 0 ); // (1) false
alert( null == 0 ); // (2) false
alert( null >= 0 ); // (3) true
```

Kyu? `>`, `<`, `>=`, `<=` `null` ko `0` me convert karte hain, isliye (3) `true` aur (1) `false`. Lekin `==` me `null` sirf `undefined` ke barabar hota hai, kisi aur ke nahi, isliye (2) `false`.

`undefined` ko kisi se compare mat karo

javascript

```
alert( undefined > 0 ); // false
alert( undefined < 0 ); // false
alert( undefined == 0 ); // false
```

- `>` aur `<`: `undefined` → `NaN`, aur `NaN` har comparison me `false` deta hai.
- `==`: `undefined` sirf `null` aur `undefined` ke barabar hai.

Problems se kaise bachein

- `undefined/null` ke saath `===` ke alawa koi bhi comparison **bahut dhyan se** karo.
- Jis variable me `null/undefined` ho sakta hai, uske saath `>`, `<`, `>=`, `<=` **mat use karo** (jab tak pakka na ho). Unhe **alag se check** karo.

Summary

- Comparison operators **boolean** return karte hain.
- Strings **dictionary order** me letter-by-letter compare hoti hain.
- Alag types ko compare karne par wo **numbers me convert** hote hain (strict equality ko chhodkar).
- `null` aur `undefined`, `==` se sirf **khud ke aur ek dusre ke** barabar hain, kisi aur ke nahi.
- `null/undefined` wale variables ke saath `>`/`<` careful use karo. Alag se check karna achha idea hai.

Practice Task (Answers)

javascript

```
5 > 4 // true
"apple" > "pineapple" // false ("a" chhota hai "p" se)
"2" > "12" // true (pehla char "2" bada hai "1" se)
undefined == null // true
undefined === null // false
null == "\n0\n" // false (null sirf undefined ke barabar)
null === +"\n0\n" // false (alag types)
```

Yaad rakhne wali baat

Hamesha `===` aur `!==` use karo. Isse type conversion ke chakkar wale bugs se bacha ja sakta hai.